

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-adb543a0234cfc5a.jsonl`  
- SHA-256: `340e9f65e35cbb32874a5b289afd8ac150472dcd161e42facf02a69acacf4973`  
- Source modified: `2026-06-10T06:13:51+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T06:12:08.993Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

ROLE: adversarial verifier. An auditor ("docs-technical") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.

FINDING: {"title":"TRACKNET_WSL_SETUP.md presents WASB custom-video inference as unbuilt future work ? it shipped","severity":"high","area":"docs/TRACKNET_WSL_SETUP.md","file":"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\TRACKNET_WSL_SETUP.md:113-118","evidence":"Doc says: 'WASB's repo documents dataset evaluation..., not custom-video inference. To run it on arbitrary match video you'd wrap its detector + temporal-consistency step in a small inference script that emits the same Frame,Visibility,X,Y CSV..., then add a WASB variant of TrackNetRunner.' Code reality: that wrapper exists and is tested ? backend/pipeline/detectors/wasb_infer.py (staged into WSL by WasbRunner, wasb_runner.py:114-131, emits <stem>_wasb.csv), backend/pipeline/detectors/wasb_runner.py:WasbRunner, and the registered wasb_hybrid segmenter (backend/pipeline/segmenters/wasb_hybrid.py). README Q7 (line 219) sends wasb_hybrid users to this doc for 'the WSL WASB setup', which never documents the implemented path or its config. Lines 52-53 also cite pre-#40 paths `backend/indexer/tracknet\_hybrid.py` / `backend/detectors/tracknet\_runner.py`, and line 4 cites `docs/DECISIONS.md` (actual: docs/archives/decisions/DECISIONS.md).","why_it_matters":"A fresh engineer would re-implement an existing, tested subsystem. This is the doc README designates as THE setup guide for the wasb_hybrid segmenter."}

Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:12:15.535Z)

```
1      # TrackNet / WASB on WSL2 ? setup & usage
2
3      Self-hosted shuttle-trajectory detectors run inside WSL2 (Ubuntu) and are bridged
4      to the Windows indexer via a subprocess adapter. See `docs/DECISIONS.md` (ADR-001)
5      for the why.
6
7      ## What's installed (WSL2 / Ubuntu 24.04)
8
9      | Thing | Location (WSL) |
10     |-----|-----|
11     | Miniconda | `~/miniconda3` |
12     | TrackNetV4 repo | `~/models/TrackNetV4` (https://github.com/TrackNetV4/TrackNetV4) |
13     | TrackNetV4 conda env | `TrackNetV4` (Python 3.9, TensorFlow 2.17 + bundled CUDA 12.3)
14     |
15     | WASB repo | `~/models/WASB-SBDT` (https://github.com/nttcom/WASB-SBDT) |
16     | WASB conda env | `wasb` (Python 3.8, torch 1.11.0+cu113) ? verified
17     | `torch.cuda.is_available()==True` |
18     | Staging dir for videos | `~/clips` (avoids slow `/mnt/c` per-frame reads) |
19
20     GPU: RTX 2070 Super visible via `nvidia-smi` inside WSL (CUDA-on-WSL working).
21
22     Activate an env with:
23     ```bash
24     source ~/miniconda3/etc/profile.d/conda.sh
25     conda activate TrackNetV4 # or: conda activate wasb
26     ```
27
28     ## ?? Two upstream issues you must resolve before TrackNetV4 inference
29
30     1. **No pretrained weights are hosted.** The "Download" links in
31     `~/models/TrackNetV4/docs/RESULT.md` are placeholders (`(#)`). You will obtain
32     weights by **training** (`src/train.py`) on your annotated data ? which is the
33     plan. Until you have a `.h5`/`.keras` weights file, the segmenter's healthcheck
34     passes but `run_predict` will log an error that `weights_path` is unset.
35
36     2. **`predict.py` has a custom-objects bug.** `src/predict.py` references layers in
37     `custom_objects` that it never imports (`MotionIncorporationLayerV1`,
38     `MotionIncorporationLayerV2`, `CombineOutputs`, `MotionFramesInput`) ? it will
39     raise `NameError` on `load_model`. The imported layers are
40     `MotionPromptLayer`, `FusionLayerTypeA`, `FusionLayerTypeB`. Patch the
41     `custom_objects` dict in `main()` to match the layers your trained model
42     actually uses, e.g.:
43     ```python
44     model = load_model(model_weights_path, custom_objects={
45         'MotionPromptLayer': MotionPromptLayer,
46         'FusionLayerTypeA': FusionLayerTypeA, # if you trained TrackNetV4_TypeA
47         'FusionLayerTypeB': FusionLayerTypeB, # if you trained TrackNetV4_TypeB
48         'custom_loss': custom_loss,
49     })
50     ```
51
52     ## Wiring it into the indexer
53
54     The segmenter is `tracknet_hybrid` (`backend/indexer/tracknet_hybrid.py`), backed by
55     `backend/detectors/tracknet_runner.py`. Configure it in `config.json` under
56     `indexing.tracknet`:
57
58     ```jsonc
59     "tracknet": {
60         "wsl_distro": "Ubuntu",
61         "repo_dir": "~/models/TrackNetV4",
62         "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
63         "conda_env": "TrackNetV4",
64     }
```

```

62     "weights_path": "",           // <-- set to the WSL path of your trained weights
63     "queue_length": 5,
64     "stage_in_wsl": true,         // copy the video into ~/clips before inference
65     "wsl_stage_dir": "~/clips",
66     "timeout_sec": 0,             // 0 = no timeout
67     "velocity_threshold": 0.02    // normalised shuttle displacement/sec to count as "in
flight"
68 }
69 ...
70
71 Then select it as the segmenter (e.g. set `indexing.default_segmenter` to
72 `tracknet_hybrid`, or pass it through whatever segmenter-selection your run uses).
73
74 ### Pipeline shape (mirrors `yolo_hybrid`)
75
76 1. **Phase 1 ? healthcheck:** activates the WSL env, confirms TF imports and a GPU
77    is visible. Fails fast with a clear message otherwise.
78 2. **Phase 2 ? TrackNet:** runs `predict.py` on the whole video, parses the
79    `Frame,Visibility,X,Y` CSV, and converts the shuttle trajectory into candidate
80    rally windows (visible + moving above `velocity_threshold`, grouped by
81    `dead_time_merge_gap`). Raw candidates are persisted for fine-tuning (same table
82    as YOLO candidates).
83 3. **Phase 3 ? AI handoff:** each candidate window is sent to the configured AI
84    provider for semantic boundary confirmation (identical to `yolo_hybrid`).
85 4. **Phase 4 ? DB:** confirmed segments written and linked back to their candidate.
86
87 ### Quick manual smoke test (inside WSL, once you have weights)
88
89 ```bash
90 source ~/miniconda3/etc/profile.d/conda.sh && conda activate TrackNetV4
91 cd ~/models/TrackNetV4/src
92 cp /mnt/c/Users/avidu/Projects/badminton-highlight-indexer/<clip>.mp4 ~/clips/
93 python predict.py --video_path ~/clips/<clip>.mp4 \
94     --model_weights <your_weights> --output_dir ~/clips/out
95 head ~/clips/out/<clip>_predict.csv # expect: Frame,Visibility,X,Y
96 ```
97
98 ## WASB (multi-sport ? and the faster path to a baseline)
99
100 **WASB ships pretrained weights**, including badminton, in
101 `~/models/WASB-SBDT/pretrained_weights/`:
102
103 - `wasb_badminton_best.pth.tar` (the WASB model)
104 - plus `tracknet2_badminton_best.pth.tar`, `monotrack_badminton_best.pth.tar`,
105   `restracknet2_badminton_best.pth.tar`, `ballseg_badminton_best.pth.tar`,
106   `deepball*_badminton_best.pth.tar` ? WASB re-implements all baselines.
107
108 This means **WASB can detect the shuttle on badminton video today**, whereas
109 TrackNetV4 has no hosted weights and must be trained first. If you want a working
110 trajectory baseline before your fine-tuning data is ready, WASB is the quicker
111 route.
112
113 Caveat: WASB's repo documents dataset *evaluation* (`main.py --config-name=eval
114 ...`), not custom-video inference. To run it on arbitrary match video you'd wrap
115 its detector + temporal-consistency step in a small inference script that emits
116 the same `Frame,Visibility,X,Y` CSV the runner already parses, then add a WASB
117 variant of `TrackNetRunner`. The trajectory?candidate-window logic is shared and
118 already tested.
119

```

3. user (2026-06-10T06:12:16.800Z)

```

backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\wasb_runner.py

```

4. assistant (2026-06-10T06:12:27.249Z)

The doc matches the quoted text. Now let me verify the code reality claims.

5. user (2026-06-10T06:12:28.678Z)

```
1      """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4          1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5          2. copies wasb_infer.py into the WASB repo's src/ dir,
6          3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7          4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9      So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10     with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11     """
12     import os
13     import shlex
14     import logging
15     import subprocess
16     from dataclasses import dataclass
17     from typing import Dict, Any, Optional, Tuple
18
19     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20     # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
21     TrackNetRunner.
22     from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
23     from backend.pipeline.detectors.base import DetectorRunner
24
25     logger = logging.getLogger(__name__)
26
27     # Windows path to the inference wrapper we stage into WSL.
28     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
29
30     @dataclass
31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
33         indexing.wasb)."""
34         distro: str = "Ubuntu"
35         repo_dir: str = "~/models/WASB-SBDT"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "wasb"
38         weights_path: str = "~/models/WASB-
39         SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
40         sport: str = "badminton"
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 0 # 0 = no timeout
43
44     @classmethod
45     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
46         w = (idx_cfg or {}).get("wasb", {}) or {}
47         return cls(
48             distro=w.get("wsl_distro", "Ubuntu"),
49             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
50             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
51             conda_env=w.get("conda_env", "wasb"),
52             weights_path=w.get("weights_path",
53                               "~/models/WASB-
54                               SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
55             sport=w.get("sport", "badminton"),
56             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(w.get("timeout_sec", 0)),
58         )
```

```

56
57
58 class WasbRunner(DetectorRunner): # DetectorRunner ABC (finish item 6 / new item 1)
59     def __init__(self, cfg: WasbConfig):
60         self.cfg = cfg
61
62     def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
63         full = f"source {self.cfg.conda_sh} && {inner_cmd}"
64         argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
65         logger.debug("WSL exec: %s", full)
66         return subprocess.run(argv, capture_output=True, text=True,
67                               timeout=(self.cfg.timeout_sec or None))
68
69     def healthcheck(self) -> Tuple[bool, str]:
70         """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
71         cmd = (f"conda activate {self.cfg.conda_env} && "
72               f"python -c \"import torch; print('torch', torch.__version__, "
73               f"'cuda', torch.cuda.is_available())\"")
74         try:
75             res = self._wsl_bash(cmd)
76         except FileNotFoundError:
77             return False, "`wsl` not found ? Windows + WSL2 required."
78         except subprocess.TimeoutExpired:
79             return False, "WSL healthcheck timed out."
80         out = (res.stdout or "").strip()
81         if res.returncode != 0:
82             return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
83         return True, out
84
85     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
86         src_mnt = to_wsl_mnt_path(video_win_path)
87         dest = f"{self.cfg.wsl_stage_dir}/{base}"
88         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)} "
89         {shlex.quote(dest)} && echo OK"
90         try:
91             res = self._wsl_bash(cmd)
92         except subprocess.TimeoutExpired:
93             logger.error("Staging video into WSL timed out.")
94             return None
95         if res.returncode != 0 or "OK" not in (res.stdout or ""):
96             logger.error("Failed to stage video into WSL: %s", (res.stderr or
97 res.stdout).strip())
98             return None
99         return dest
100
101     def _stage_infer_script(self) -> bool:
102         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
103         src_mnt = to_wsl_mnt_path(_INFER_WIN)
104         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
105 OK"
106         res = self._wsl_bash(cmd)
107         return res.returncode == 0 and "OK" in (res.stdout or "")
108
109     def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
110         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or
111 None."""
112         os.makedirs(output_win_dir, exist_ok=True)
113         # Normalize for cross-platform basename (tests use Windows paths on Linux).
114         video_win_path = str(video_win_path).replace("\\", "/")
115         base = os.path.basename(video_win_path)
116         stem = os.path.splitext(base)[0]
117         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
118         expected_csv_win = os.path.join(output_win_dir, f"{stem}_wasb.csv")
119
120         if not self._stage_infer_script():

```

```

117         logger.error("Could not stage wasb_infer.py into WSL.")
118         return None
119     staged_video = self._stage_video(video_win_path, base)
120     if staged_video is None:
121         return None
122     frames_dir = f"{self.cfg.wsl_stage_dir}/{stem}_frames"
123
124     cmd = (
125         f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
126         f"python wasb_infer.py "
127         f"--video {shlex.quote(staged_video)} "
128         f"--frames_out_dir {shlex.quote(frames_dir)} "
129         f"--weights {shlex.quote(self.cfg.weights_path)} "
130         f"--sport {shlex.quote(self.cfg.sport)} "
131         f"--out {shlex.quote(wsl_out_dir + '/' + stem + '_wasb.csv')}}")
132     )
133     logger.info("Running WASB inference on %s ...", base)
134     try:
135         res = self._wsl_bash(cmd)
136     except subprocess.TimeoutExpired:
137         logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
138         return None
139     if res.returncode != 0:
140         logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])

```

6. user (2026-06-10T06:12:29.708Z)

```

backend\pipeline\segmenters\__init__.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py

```

7. user (2026-06-10T06:12:30.689Z)

219:[Omitted long matching line]

8. user (2026-06-10T06:12:37.293Z)

```

210     python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter gemini
211
212     # Pure CV baseline ? no API key needed
213     python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter motion
214     ```
215     *Or set `"default_segmenter": "yolo_hybrid"` (or `"gemini"`/`"motion"`) under
`"indexing"` in `config.json`.
216
217     The AI provider used by `yolo_hybrid` and `gemini` is controlled by `ai_provider` +
`ai_model` at the top level of `config.json`. Today only `"gemini"` is registered; new providers
can be added under `backend/providers`.
218
219     **Trajectory-based segmenters (`wasb_hybrid`, `tracknet_hybrid`).** The registry also
includes two shuttle-trajectory variants. Like `yolo_hybrid`, they cheaply filter the video down
to candidate windows *locally* and then route each window to the AI provider (`gemini`) for
precise boundaries ? so they need the same `GEMINI_API_KEY`. **If an observability report shows
`segmenter_actual: wasb_hybrid` (or `tracknet_hybrid`), that is a valid, registered segmenter ?
not an error.** Pick one with `--segmenter wasb_hybrid` or via `default_segmenter`.
(`wasb_hybrid` additionally requires the WSL WASB setup ? see
[docs/TRACKNET_WSL_SETUP.md](docs/TRACKNET_WSL_SETUP.md).)
220
221     ### Q8: How does the `yolo_hybrid` pipeline actually save money?

```

222 Sending a multi-gigabyte video to a hosted LLM is slow and expensive. `yolo\_hybrid` runs
in four phases:
223
224 1. **Chunking** ? splits the video into overlapping windows (`chunk\_duration`,
`chunk\_overlap`) so we can process incrementally.
225 2. **Detection + tracking** ? torchvision person detection (Faster R-CNN) + ByteTrack
runs on each chunk locally (GPU-accelerated when available), measuring per-track velocity.
Frames whose normalised player velocity exceeds `yolo\_velocity\_threshold` are kept; the rest are
dropped. Adjacent active frames are merged into candidate windows, with
sub-`min\_action\_window\_duration` windows discarded.
226 3. **AI handoff** ? only the surviving candidate windows (padded by `ai\_handoff\_padding`
seconds on each side) are extracted and sent to the AI provider for precise rally boundaries and
shot-count estimation. These calls run in parallel up to `ai\_max\_workers`.
227 4. **DB population** ? confirmed segments are written to `play\_segments` along with
`shot\_count`, `shot\_count\_confidence`, and a `detector` tag like `yolo-hybrid-gemini-2.5-flash`.

9. user (2026-06-10T06:12:38.321Z)

docs\archives\decisions\DECISIONS.md

10. user (2026-06-10T06:12:39.295Z)

No files found

11. user (2026-06-10T06:12:40.021Z)

Found 6 files
tests\test_rally_slicer.py
tests\test_tracknet_runner.py
tests\test_wasb_cache.py
tests\test_reporting_harvest.py
tests\test_export_wasb_segments.py
tests\test_calibrate_wasb.py

12. assistant (2026-06-10T06:12:58.040Z)

All core claims verified. Let me check counter-evidence ? whether CODE_MAP.md or other docs
already document the implemented WASB path (which would mitigate), and confirm the segmenter
registration.

13. user (2026-06-10T06:13:00.059Z)

38:Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same shape):
40:P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
48:### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
50:WasbHybridSegmenter.process_video @backend/pipeline/segmenters/wasb_hybrid.py
51: -> WasbRunner.healthcheck() (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
52: -> WasbRunner.run_predict(video, out_dir)
53: stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
54: -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
55: -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
58:(`tracknet\_hybrid` identical, swapping WasbRunner->TrackNetRunner; `\_probe\_fps\_width`
fallback (30.0, 1280.0).)
62:GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts -> List[Interval]
68:Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate} @backend/eval/calibration.py
74:Entrypoints: `@run\_eval.py` (single video score), `@run\_phase3\_eval.py` (manifest batch, can
segment), `@run\_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate\_wasb.py` + `@backend/eval/export\_wasb\_segments.py` (WASB tuning
drivers).

```
81:- `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries may still use `print` for direct output.
```

```
109:- `@backend/pipeline/segmenters/__init__.py` ? imports all 5  
(`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid`) = the de-facto registration manifest; re-exports them + `segmenter_registry` via `__all__`.
```

```
110:[Omitted long matching line]
```

```
111:- `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name `tracknet_hybrid`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig` from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`; `detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably yolo_hybrid).
```

```
121:[Omitted long matching line]
```

```
122:[Omitted long matching line]
```

```
123:[Omitted long matching line]
```

[Showing results with pagination = limit: 20]

14. user (2026-06-10T06:13:01.567Z)

```
58:         sport_name = self.config.get("sport", "badminton")  
66:         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))  
67:         model_name = self.config.get("ai_model", idx_cfg.get("ai_model", "gemini-2.5-flash"))  
225:segmenter_registry.register(WasbHybridSegmenter)
```

15. user (2026-06-10T06:13:02.196Z)

Found 1 file

tests\test_tracknet_runner.py

16. user (2026-06-10T06:13:18.949Z)

docs\EVALUATION.md:21:> bottleneck. See `docs/TRACKNET\_WSL\_SETUP.md`.

docs\README.md:18:- TRACKNET_WSL_SETUP.md

17. user (2026-06-10T06:13:23.482Z)

```
10      - [DEFERRED_HARDENING_PLAN.md](DEFERRED_HARDENING_PLAN.md) ? Design (pending approval) for the larger deferred items: packaging, train/eval guardrails, and the async job model (#16, the first infra slice toward the platform).
```

```
11      - [BACKLOG.md](BACKLOG.md) ? Deferred items with rationale and revisit triggers.
```

```
12      - [ROADMAP.md](ROADMAP.md) ? High-level four-phase product direction.
```

```
13      - [VIDEO_RECORDING_GUIDELINES.md](VIDEO_RECORDING_GUIDELINES.md) ? Capture best practices (referenced by reports).
```

```
14      - [HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md) ? Mental model of the two-layer detection system.
```

```
15      - [EVALUATION.md](EVALUATION.md) ? How we measure and improve quality.
```

```
16
```

```
17      ## Practical / ongoing
```

```
18      - [TRACKNET_WSL_SETUP.md](TRACKNET_WSL_SETUP.md)
```

```
19      - Golden set docs (`GOLDEN_SET`)
```

```
20      - Evaluation harness and calibration drivers (in `backend/eval/`)
```

```
21
```

```
22      ## Historical documents
```

```
23      Completed plans, old research, ADRs, and checkpoints live in [`archives/`](archives/).
```

```
24
```

18. user (2026-06-10T06:13:46.421Z)

Structured output provided successfully

19. assistant (2026-06-10T06:13:51.430Z)

Verification complete.